Full Stack Development With MERN

1. Introduction

Project Title: Book A Doctor

Online Doctor Appointment Booking System

Project Type: Full Stack Development With MERN

Team Members:

Team Leader-Shanmukha Venkata Vardhan Thatavarthy

Backend Developer

Team Member-Yemineni Kavya

Frontend Developer

2. Project Overview:

Purpose:

- To provide patients with a simple and efficient platform to book appointments online.
- To reduce waiting times and improve accessibility by allowing users to search, schedule, and manage appointments digitally.
- To help doctors and clinics organize their schedules and manage patient bookings seamlessly.
- To create a bridge between patients and healthcare providers, ensuring better communication and convenience.

Goals:

- User-Friendly Interface
- Appointment Booking System
- Authentication & Security
- Doctor Dashboard
- Database Integration
- Scalability
- Optional Enhancements

Features:

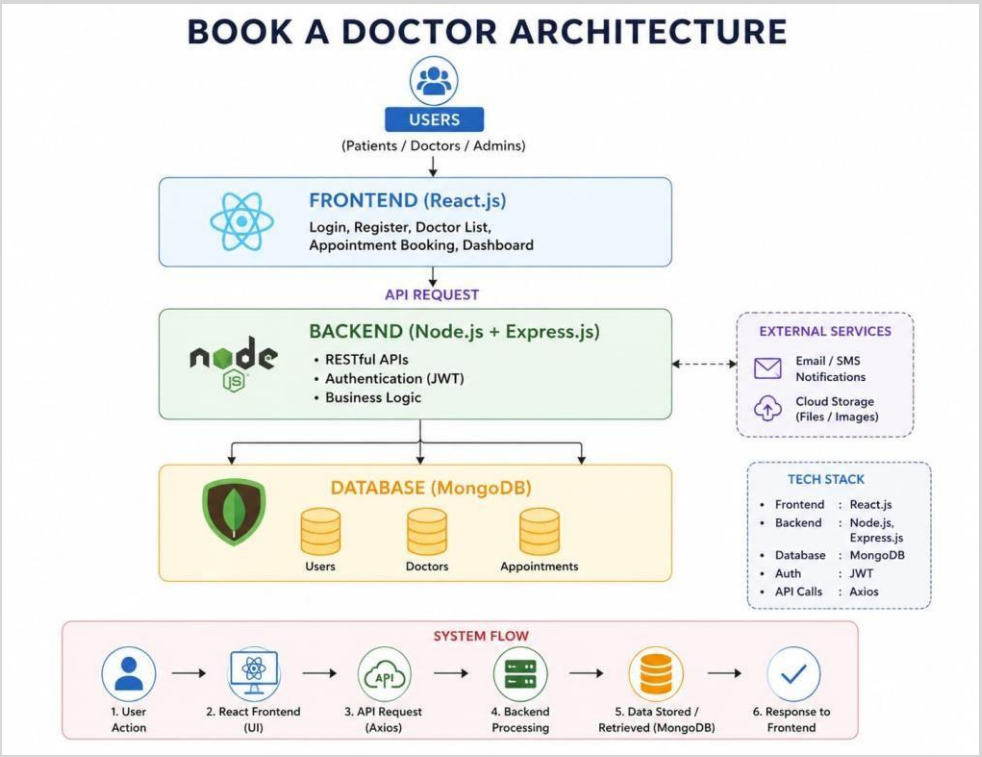
- U**ser Registration & Login** – Secure authentication for patients and doctors.
- **Doctor Profiles** – Detailed information including specialization, experience, and availability.
- **Search & Filter** – Patients can search doctors by name, specialty, or location.
- **Appointment Booking** – Easy scheduling with available time slots.
- **Doctor Dashboard** – Doctors can view, accept, or manage appointments.
- **Patient Dashboard** – Patients can track upcoming and past appointments.
- **Notifications & Reminders** – Email/SMS alerts for appointment confirmations and reminders.
- **Medical History Tracking** – Patients' past consultations stored for reference.
- **Payment Integration** – Online payment for consultation fees.
- **Responsive Design** – Works smoothly across desktop, tablet, and mobile devices.
- **Admin Panel (optional)** – Manage doctors, patients, and system settings.

3. Architecture

Frontend (React): The frontend is built using React.js, providing a responsive and dynamic user interface. It uses reusable components for pages like login, doctor profiles, appointment booking, and dashboards. State management is handled with React hooks or Redux, and API calls are made via Axios/Fetch to communicate with the backend.

Backend (Node.js + Express.js): The backend is developed with Node.js and Express.js, exposing RESTful APIs for authentication, appointment booking, and user management. Middleware handles validation, error handling, and security (JWT authentication).

Database (MongoDB): MongoDB stores collections for users (patients/doctors), appointments, and medical history. Relationships are managed via references (e.g., patient ID linked to appointment). Mongoose schemas define the structure and enforce validation.



4. Setup Instructions:

- **Prerequisites:**
  - Node.js (latest LTS version)
  - MongoDB (local or Atlas cloud instance)
  - Git
- **Installation:**
  1. Clone the repository: git clone https://github.com/shanmukh2396/book_a_doctor.git
  2. Navigate the project folder
  3. Install dependencies for both client and server: cd client, npm install, cd ../server, npm install.
  4. Set up environment variables (.env file) for MongoDB URI, JWT secret, and server port.

5. Folder Structure:

1. **Client (React Frontend):** client/
   src/
   components/
   pages/
   services/
   App.js
   index.js
2. **Server (Node.js Backend):** server/
   controllers/
   models/
   Routes/
   middleware/
   server.js

6. Running the Application:

1. **Frontend:** cd frontend
   npm start
2. **Backend:** cd backend
   npm start

7. API Documentation:

1. **User Authentication**

- POST /api/auth/register → Register new user
- POST /api/auth/login → Login user

2. **Doctor Management**

- GET /api/doctors → Get list of doctors
- GET /api/doctors/:id → Get doctor details

3. **Appointments**

- POST /api/appointments → Book appointment
- GET /api/appointments/: userId → Get user's appointments

8. Authentication:

- Implemented using **JWT (JSON Web Tokens)**.
- On login, a token is generated and stored in local storage.
- Protected routes require token validation via middleware.
- Role-based access (patient vs doctor) ensures proper authorization.

9. User Interface:

- Patient dashboard for booking and tracking appointments.
- Doctor dashboard for managing schedules.
- Search and filter functionality for finding doctors.
- Responsive design for desktops and mobile.

10. Testing

- **Unit Testing:** Jest/Mocha for backend APIs.
- **Integration Testing:** Postman collections for API endpoints.
- **Frontend Testing:** React Testing Library for component testing.

11. Screenshots or Demo:

The Demo videos are as follows.
https://drive.google.com/drive/folders/1Yd1q_VfczGm0ha73jWdwgtDL49_zi8ZE?usp=drive_link

12. Knowing Issues:

- Limited errors handling invalid appointment times.
- No integrated payment gateway yet.
- Notifications only via email (SMS not implemented).

13. Future Enhancements:

- Telemedicine integration for online consultations.
- AI-based doctor recommendations.
- Multi-language support.
- Payment gateway integration.
- Advanced analytics for doctors and clinics.